感悟报告

1. linux真是一切皆文件，之前预习的时候还没觉得有什么，实际上手尝试做题后发现，无论是linux的环境还是日志的表现形式都是以文件的格式存在的，就连报错日志都需要追加到日志里，而不是单纯打印到屏幕上，无论是这次考核要求的通关形式还是网上的相关视频，Linux环境中总是严格按文件形式分好类，格式干净整洁有序
2. 之前预习时背下来了linux的相关命令，但实际上手发现远没有那么简单，无论是查看还是操作，都会有更合适的命令在里面，如各种减号后缀代表的各种意思
3. ai的使用是这次学习最大的占比了，也让我深刻意识到ai使用并不是一项无用的技术，反而是保证对话高效有序进行的一门学问，诸如让语言变得通俗易懂，根据自身水平建立讲解模式，还是代码讲解的表现形式，用好ai都能节省大量的时间精力，就比如因为我给ai的截图不清晰导致中文与英文搞混排查错误又浪费了大把的时间，还是说提示词表述不清导致ai答复答非所问
4. 相关语言术语也是在这次考核中给予我颇深理解障碍的环节，就比如linux相关题目中要求我严格以二进制比对，尝试用nano直接进行修改反而无法通关，原因是nano自带的换行符，又比如shell脚本编译中那些晦涩难懂的专业术语，工作目录，接口，结果文件路径，配置文件，进程号文件，实例，瞬时资源数值，真实状态等，对我理解题目造成了很大的阻碍，让我再次意识到基础的关键不仅在于用于做题，还是用于题目理解
5. 进程之间是分离的，也是让我印象深刻的一点，无论是第四关出现的export将变量看给子进程还是hoc中第一关c语言收集信息是双开进程，都表现了进程的分离性
6. 严格的比较性，linux中有一题要求运行test，我直接使用workspace下的test.bin却无法通关，证实了文件名称的严格比较
7. 中文字节与英文字节的非一致性，在shell中一题中，我尝试grep字符然后cut切割选取我想要的字符，但是却失败了，原因就是我用中文冒号作为分隔符，占三字节，而cut只允许一字节
8. 性能优化的检验，之前我一直以为hpc是只有python，这次考核告诉我性能优化绝不止一种编程语言，在这次性能优化效能的检验中，我意识到检验性能的绝不止一种指标，除去python中的加速比和并行效率，还有平均时间，中位数，总体标准差，计算吞吐，gflops，有效带宽等，threads的不同也会有不同的实验效果，可能存在反直觉的性能结果
9. c和python的算法优化相似性，刚接触c语言的性能优化，我是晕头转向，但在比较有意深入了解后，发现与python也是有许多相似性的地方，比如多进程，矩阵乘法等，这帮助我更快的上手c语言的性能优化，即便是在c基础还在打情况下
10. 无论是shell，linux还是hpc性能优化，都考眼了对逻辑思维的严格考察，从在与ai问答反复提及“这串代码想表达的意思，在整串代码中的作用”就能看出来逻辑的严格重要性，比如一道shell脚本的题要求在src文件夹里cmake，我放在了build里，理所当然的报错了，cd的位置，cmake的对象状态都给予了我很大启发
11. 信息的收集，无论是读懂题目，看懂脚本，信息的收集都至关重要，再次拿第十点的例子打比方，错误的主要原因其实还是信息收集的不完全，这启示我在做题前，要京可能的收集给的一切信息，无论是cat查看cmakelist文件还是--help查看函数或程序的使用规则和传入参数，都会比报错后查看报错信息要来的高速快捷，这也同样启示使用ai要给他喂足够量的信息，就比如hpc第三题，没有足够的信息，ai是无法解出性能优化方案的
12. git的团队协作，从前我一直觉得团体写代码一直很高大上并且遥不可及，但实际体验后发现，其实每次修改都有迹可循，修改冲突也是最朴实无华的重新覆写，让我减轻了对git协作方面的困惑，有协作的动力

这次考核给了我一个沉浸于逻辑语言艺术美感的机会，我花费了大量时间从一个小白成长为了大白（还是很菜）（），但也是实实在在感受到了学习这一方面的乐趣，看着这一段时间的笔记也让我颇有成就感，让我意识到兴趣也是进步的一大动力，